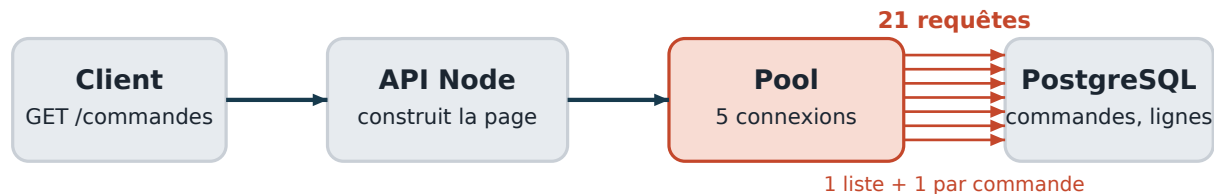


Besoin et problème initial

problème → mécanisme → mesure → décision
à lire : environ 58 s · cumul ≈ 0 min 58 s

CE QUE FAIT UNE PAGE DE 20 COMMANDES



Mesure initiale (20 clients, N+1)	Valeur
Débit terminé (plafonne dès 5 clients)	≈ 247 requêtes/s
Latence p95	98 ms
File d'attente du pool (max)	15 requêtes
Erreurs	0

À DIRE

Bonjour. Mon cas, c'est ShopFlow, une boutique en ligne : une base PostgreSQL 18 avec 100 000 commandes et 300 000 lignes de commande. Je m'intéresse à un endpoint très simple, GET /commandes, qui renvoie une page de 20 commandes avec leurs lignes. Sur le schéma : le client appelle l'API, l'API passe par un pool de 5 connexions, puis interroge PostgreSQL. Le problème observé : pour construire une seule page, l'API envoie **21 requêtes SQL**, une pour la liste puis une par commande. Ma mesure de départ, avec 20 clients simultanés : le débit plafonne à environ **247 requêtes par seconde** dès 5 clients, le p95 est de **98 millisecondes**, et jusqu'à 15 requêtes attendent une connexion. Zéro erreur, mais le service est saturé.

Mécanisme et intervention

problème → mécanisme → mesure → décision
à lire : environ 59 s · cumul ≈ 1 min 57 s

ANCIENNE VERSION

```
rows = SELECT ... commandes
        LIMIT 20;
POUR CHAQUE commande :
  SELECT ... FROM lignes
    WHERE commande_id = $1;

-- 1 + 20 = 21 requêtes
```



NOUVELLE VERSION

```
rows = SELECT ... commandes
        LIMIT 20;
SELECT ... FROM lignes
  WHERE commande_id =
    ANY($1::bigint[]);
-- $1 = les 20 ids
-- 2 requêtes, toujours
```

OÙ EST LE COÛT ? (TEMPS PAR REQUÊTE DE LIGNES)

vu par PostgreSQL

0,018 ms

vu par l'API (aller-retour)

0,68 ms

×38 : le coût est dans les allers-retours

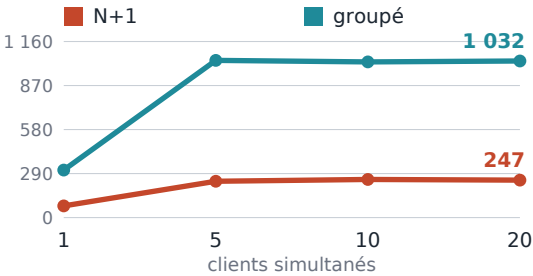
À DIRE

Première question : où est le coût ? J'ai regardé pg_stat_statements. La requête de lignes apparaît avec 20 000 appels de 0,018 milliseconde : 369 millisecondes cumulées, elle passe inaperçue, et elle ne figure pas non plus dans le journal des requêtes lentes. Mais l'API, elle, mesure 14,2 millisecondes de SQL par page : environ 0,68 milliseconde par requête, soit **38 fois plus** que ce que voit PostgreSQL. Donc le coût n'est pas dans la requête, il est dans les **allers-retours**. Mon hypothèse causale : si je charge toutes les lignes de la page en une seule requête, avec ANY et la liste des identifiants, je passe de 21 à 2 requêtes. C'est **la seule chose que je change**, comme on le voit sur le code à gauche.

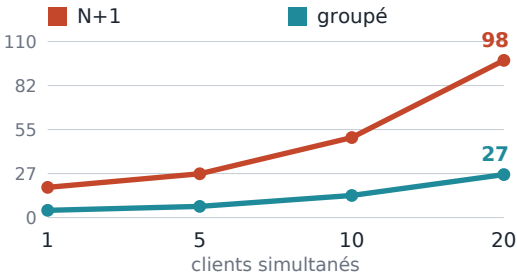
Mesures avant / après

problème → mécanisme → mesure → décision
à lire : environ 58 s · cumul ≈ 2 min 55 s

Requêtes terminées par seconde



Latence p95 (ms)



20 clients, médiane de 3 essais	Avant (N+1)	Après (groupé)	Gain
Débit (req/s)	246,9	1032,1	×4,2
p95 (ms)	98,0	26,8	−73 %
Erreurs	0	0	=
Les 3 essais (req/s)	238 à 253	978 à 1082	intervalles disjoints

À DIRE

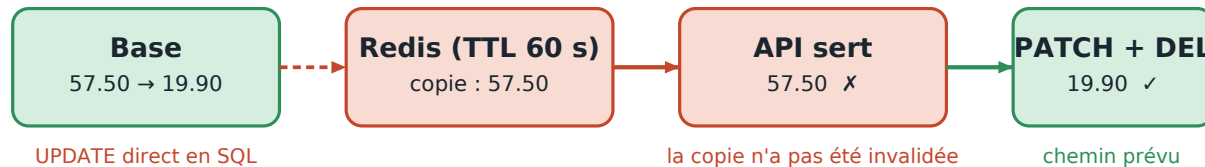
Pour mesurer, je garde les mêmes conditions : même machine, même jeu de données, 1, 5, 10 et 20 clients, trois essais alternés. Sur la courbe de gauche, le débit du N+1 reste plat autour de 250 dès 5 clients : ajouter des clients n'ajoute que de l'attente. Le groupé monte et se stabilise bien plus haut. À 20 clients, le débit passe de 247 à **1032 requêtes par seconde**, soit fois 4,2. Sur la courbe de droite, le p95 passe de 98 à **26,8 millisecondes**, moins 73 pour cent. Zéro erreur. Les trois essais ne se chevauchent pas : c'est reproductible. Et le processeur reste à 50 pour cent dans les deux cas : le gain vient du travail évité, pas d'une ressource en plus.

Correction et fraîcheur

problème → mécanisme → mesure → décision
à lire : environ 65 s · cumul ≈ 4 min 00 s

Correction (même résultat, mêmes droits)	Résultat
JSON complet N+1 = groupé (5, 20, 50, 100 commandes)	identique
Sans jeton / client_id dans l'URL / curseur falsifié	401 / 400 / 400

FRAÎCHEUR : LE CACHE PEUT MENTIR



Cache : miss 2,7 ms (1 SELECT) → hit 0,99 ms (0 SQL) ; fraîcheur acceptée ≤ 60 s ; jamais de cache sur l'achat.

À DIRE

Je ne regarde pas que la vitesse : un résultat plus rapide mais faux ne sert à rien. Le JSON complet, commandes et lignes, est **identique** en N+1 et en groupé, pour 5, 20, 50 et 100 commandes, et les droits sont conservés : 401 sans jeton, 400 si on injecte un client dans l'URL. Ensuite, la fraîcheur, avec le cache Redis sur la fiche produit : un hit ne fait aucune requête SQL, un miss en fait une. Mais regardez : après une modification faite directement dans la base, l'API a continué de servir **57.50 alors que la base disait 19.90**. Le TTL de 60 secondes borne la copie périmée, il ne la corrige pas. Quand on passe par l'API avec un PATCH, la copie est invalidée et la valeur redevient juste. Et l'achat ne passe jamais par le cache.

Décision, variante rejetée, risque

problème → mécanisme → mesure → décision
à lire : environ 72 s · cumul ≈ 5 min 12 s

	Décision et raison mesurée
Retenu	chargement groupé : 21 → 2 requêtes, ×4,2 de débit, p95 –73 %, 0 erreur
Rejeté	agrandir le pool 5 → 20 : ≈ 287 req/s contre 1032 en groupé (2 essais, indicatif) ; PgBouncer : 40 → 5 connexions mais 261 → 94 tps
Risque restant	copie périmée jusqu'à 60 s ; course entre une lecture lente et une invalidation (reproduite à la main)
Portée	cette machine (8 cœurs, WSL2), ce jeu (100 000 commandes), 1 à 20 clients ; non testé en production ni avec plusieurs instances

3 CONCLUSIONS

1. J'ai observé 21 requêtes par page et un débit plafonné.
2. En les ramenant à 2, le débit est ×4,2, sans erreur.
3. Valable ici (machine, jeu, 1–20 clients) ; non testé en production.

À DIRE

Ma décision : je retiens le chargement groupé. Une variante rejetée : agrandir le pool de 5 à 20 connexions. Cela supprime l'attente, mais n'atteint que 287 requêtes par seconde, contre 1032 avec le groupé, sur deux essais seulement, donc à titre indicatif. J'ai aussi testé PgBouncer : il fait passer les connexions de 40 à 5, mais le débit tombe de 261 à 94 transactions par seconde. Il limite les connexions, il n'accélère rien. Risque restant : la copie du cache peut être périmée jusqu'à 60 secondes, et il existe une course entre une lecture lente et une invalidation, que j'ai reproduite à la main. Pour conclure en trois phrases : j'ai observé 21 requêtes par page et un débit plafonné ; en les ramenant à 2, le débit est multiplié par 4,2 sans erreur ; c'est valable sur cette machine, ce jeu et 1 à 20 clients, et pas encore démontré en production.

Si on me pose la question

reproductibilité · conséquence pour l'API
à lire : environ 0 s · cumul ≈ 5 min 12 s

Question probable	Réponse courte
Comment reproduire sur un autre poste ?	même base (01_schema.sql, 02_donnees.sql, md5 comparés), PostgreSQL 18.6, Node 22, pool 5 ; charge fermée 1/5/10/20 clients, 3 essais alternés N+1 / groupé ; python3 atelier9/p2_charge.py (≈ 16 min) ; on compare l'ordre de grandeur
Que change la décision pour l'API ?	contrat inchangé (même JSON, mêmes droits) ; 21 → 2 requêtes donc moins de connexions occupées ; tableau d'ids borné à 100 ; avec le cache : copie ≤ 60 s et repli sur PostgreSQL si Redis tombe
Et si le gain disparaît ?	je le dirais : les séries pool et charge ouverte étaient instables, je les ai présentées essai par essai
Écart de protocole ?	ma mesure : 3 s d'échauffement et 10 s par essai ; la fiche cite 10 s et 30 s en exemple

À DIRE

Reproductibilité : même base, mêmes versions, même protocole ; je compare l'ordre de grandeur, pas la valeur exacte.

Conséquence pour l'API : contrat inchangé, moins de connexions occupées, fraîcheur bornée si cache, repli sur PostgreSQL si Redis tombe.

Variante rejetée : agrandir le pool ou PgBouncer pour accélérer. **Risque restant** : la copie périmée du cache.